

```
C: > Users > pappu > Documents > bookflow_api > main.py > register_user

66  async def search_books(
122      for r in rows
123  ],
124  }
125
126
127  @app.post("/register", status_code=201)
128  async def register_user(user: UserRegisterSchema):
129      hashed_password = pwd_context.hash(user.password)
130
131      # In a full application, the username and hashed password
132      # would be saved to a users table.

Problems 11 Output Debug Console Terminal Ports

(venv) PS C:\Users\pappu\Documents\bookflow_api> dir
-----
d-----      12-09-2026      14:03      logs
d-----      12-09-2026      13:48      venv
-a-----      12-09-2026      13:53      501 database.py
-a-----      12-09-2026      14:03      3347 main.py
-a-----      12-09-2026      13:56      498 models.py
-a-----      12-09-2026      14:02      986 schemas.py

(venv) PS C:\Users\pappu\Documents\bookflow_api> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\pappu\\Documents\\bookflow_api']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [26932] using StatReload
INFO: Started server process [11296]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

BookFlow Gatekeeper API 0.1.0 OAS 3.1

/openapi.json

default

POST /books Create Book

GET /books/search Search Books

POST /register Register User

Schemas

BookSchema > Expand all object

HTTPValidationError > Expand all object

UserRegisterSchema > Expand all object

ValidationError > Expand all object

BookFlow Gatekeeper API

0.1.0

OAS 3.1

/openapi.json

default

POST

/books

Create Book

Parameters

No parameters

Request body

required

Edit Value

Schema

```
{
  "title": "Interstellar",
  "author": "Christopher Nolan",
  "description": "A science-fiction adventure about space, survival and humanity.",
  "isbn": "9781234567890",
  "price": 499.00,
  "published_date": "2014-11-07"
}
```

201

Response body

```
{
  "id": 6,
  "title": "Interstellar"
}
```

Response headers

```
content-length: 31
content-type: application/json
date: Sat, 12 Sep 2026 13:57:27 GMT
server: uvicorn
```

Responses

Code	Description
201	Successful Response
	Media type application/json Controls Accept header. Example Value Schema <pre>"string"</pre>
422	Validation Error
	Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>

```
PS C:\Users\pappu> cd Documents
PS C:\Users\pappu\Documents> cd bookflow_api
PS C:\Users\pappu\Documents\bookflow_api> Get-Content logs\activity.log
2026-09-12T13:57:28.606884 - New Book Added: Interstellar
PS C:\Users\pappu\Documents\bookflow_api>
```

Name	Description
q required	
string (query)	
limit	3
integer (query)	

Execute

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/books/search?q=X20&limit=3' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/books/search?q=X20&limit=3
```

Server response

Code	Details
400 Undocumented	Error: Bad Request Response body <pre>{ "detail": { "error": { "code": "INVALID_PARAMETER", "message": "'q' must not be empty." } } }</pre> Response headers <pre>content-length: 84 content-type: application/json date: Sat, 12 Sep 2026 14:05:54 GMT</pre>

```
"detail": {
  "error": {
    "code": "INVALID_PARAMETER",
    "message": "'q' must not be empty."
  }
}
```

Response headers

```
content-length: 84
content-type: application/json
date: Sat, 12 Sep 2026 14:05:54 GMT
server: uvicorn
```

Responses

Code	Description
200	Successful Response Media type application/json Controls Accept header. Example Value Schema <pre>"string"</pre>
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>

Name	Description
q required	
string (query)	
limit	0
integer (query)	
Execute	
Responses	
Curl	
<pre>curl -X 'GET' \ 'http://127.0.0.1:8000/books/search?q=%20&limit=0' \ -H 'accept: application/json'</pre>	
Request URL	
<pre>http://127.0.0.1:8000/books/search?q=%20&limit=0</pre>	
Server response	
Code	Details
400 Undocumented	Error: Bad Request
Response body	
<pre>{ "detail": { "error": { "code": "INVALID_PARAMETER", "message": "'q' must not be empty." } } }</pre>	
Response headers	
<pre>content-length: 84 content-type: application/json date: Sat, 12 Sep 2026 14:07:04 GMT server: uvicorn</pre>	

POST **/register** Register User

Parameters

No parameters

Request body required

Edit Value | Schema

```
{
  "username": "student1",
  "password": "BookFlow@123"
}
```

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/register' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "student1",
    "password": "BookFlow@123"
  }'
```

Request URL

```
http://127.0.0.1:8000/register
```

Server response

Code Details

Request URL

http://127.0.0.1:8000/register

Server response

Code	Details
201	Response body <pre>{ "username": "student1", "message": "Registered successfully"}</pre> Response headers <pre>content-length: 59content-type: application/jsondate: Sat, 12 Sep 2026 14:13:41 GMTserver: uvicorn</pre>

Responses

Code	Description
201	Successful Response Media type application/json Controls Accept header. Example Value Schema "string"
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }]}</pre>

Conclusion:

In this experiment, I successfully developed the BookFlow Gatekeeper API using FastAPI. The application validates book and user data using Pydantic, connects asynchronously to the existing PostgreSQL bookflow_db, performs semantic book searches using pgvector embeddings, and handles invalid parameters with appropriate 400 responses. I also implemented background logging for newly added books and password hashing for user registration. This experiment builds on Experiment 1 by connecting the previously created database, vector embeddings, and search design to a working FastAPI backend with validation, security, and API testing through Swagger UI.